Presentation Notes: College Registration System Database Design

SHEMA SEBATUNZI Samuel · BSCSR/2026/69594 · 21 October 2026 · About 6–8 minutes

Slide 1: Title ~30 s

> *"Good evening. I am Shema Sebatunzi Samuel. I will present the database design for a college registration system, from the requirements to the final relational tables built in Draw.io."*

Slide 2: Requirements: users & processes ~1 min

- **Goal:** let students register for the courses in their programme each semester.
- **Five users:** Student, Registrar, Lecturer, Head of Department, System Administrator.
- **Main flow:** enrol student → student registers (*Pending*) → Registrar approves (*Approved*) → lecturer enters grade (*Completed*).
- **Out of scope:** fees, timetabling, admissions.

Slide 3: Entities & business rules ~1.5 min

- **Entities:** Student, Programme, Course, Registration and Lecturer, plus **Department** to group programmes and staff.
- A student belongs to **one** programme; a course has **one** coordinating lecturer.
- A programme has many courses, and a course can be shared by many programmes (**many-to-many**).
- A student can register for a course **only once per semester**.
- A grade can be entered only when the registration is **Completed**.

> *"Each rule is enforced by the database itself, through keys and constraints, not only by the application."*

Slide 4: Conceptual model (ERD) ~1.5 min

- Rectangles are **entities**, diamonds are **relationships**, and 1 / N / M show **cardinality**.
- One-to-many: Department–Programme, Department–Lecturer, Programme–Student, Lecturer–Course.
- Two many-to-many links: **Programme–Course** and **Student–Course** (through Registration).

Slide 5: Relational schema & data types ~1.5 min

- **7 tables.** Every table has a **primary key** (yellow); foreign keys are blue.
- Each one-to-many relationship becomes a **foreign key** on the "many" side.
- Each many-to-many relationship becomes a **junction table**: PROGRAMME_COURSE and REGISTRATION.
- **Data types:** INT for keys, VARCHAR for names and codes, DATE for dates, SMALLINT for credits and semesters, BOOLEAN for flags.
- The design is normalised to **Third Normal Form**, so nothing is stored twice.

Slide 6: Implementation & checks ~1 min

- I implemented the design in SQL and tested it with sample data.
- The database **rejected** every invalid entry: a duplicate registration, an unknown student, an invalid value, an early grade, and deleting a department that is still in use.

> *"In conclusion, the design meets the requirements: every table has a primary key, the relationships are consistent, and the business rules are enforced. Thank you. I welcome your questions."*

Likely questions & short answers

Why add a Department entity?	Programmes and lecturers both belong to a department. Storing it once avoids repeating department details.
Why is REGISTRATION a separate table?	Student–Course is many-to-many. The registration table resolves it and stores the year, semester, status and grade.
What is PROGRAMME_COURSE?	A junction table for the programme curriculum. Its primary key is the pair (programme_id, course_id).
How do you stop double registration?	A UNIQUE constraint on (student, course, academic year, semester).
Why surrogate (INT) keys?	They are short, stable and fast to join. Real-world codes such as reg_number stay UNIQUE.
Is it normalised?	Yes, to 3NF. There are no repeating groups and no partial or transitive dependencies.
What happens if a department is deleted?	It is blocked (ON DELETE RESTRICT) while programmes or lecturers still depend on it.
Which DBMS does it target?	MySQL 8 or PostgreSQL. The SQL works on both.

Tip: speak slowly, point at the diagram when you name a relationship, and keep each slide to its time.